FDA / NCTR Regulatory Research Cluster (RRC)

Apptainer User Guide · Version 1.0 · August 2026

Companion document to the RRC User Guide v3.0

HPC Support · **NCTRHPCSupport@fda.hhs.gov**

1. Introduction

Apptainer (formerly Singularity) is the recommended container platform for the FDA/NCTR Regulatory Research Cluster (RRC). Unlike Docker, Apptainer is designed specifically for HPC environments: it requires no root privileges, integrates seamlessly with SLURM, and provides full GPU passthrough. Containers allow you to package your entire software environment — libraries, dependencies, and runtime — into a single portable image file (.sif), ensuring that your workflow runs identically across different nodes and cluster configurations.

This guide covers everything you need to use Apptainer effectively on the RRC: from understanding definition files and building images, to running containers interactively and in batch jobs, and managing bind paths to access cluster storage.

1.1 Why Apptainer on HPC?

Feature	Docker	Apptainer
Root privileges required	Yes	No — safe for shared HPC
SLURM integration	Limited	Native and seamless
GPU passthrough	Requires --runtime=nvidia	Built-in with --nv flag
User namespace isolation	Full daemon	Rootless, user-space
Image format	Layered (OCI)	Single .sif file — portable
Docker Hub compatible	Yes	Yes — pull and convert directly

1.2 Key Terms

Term	Definition
`SIF file`	Singularity Image Format — a single, immutable, portable container image file.
`Definition file (.def)`	A recipe file that describes how to build a container image.
`Bootstrap`	Specifies where Apptainer obtains the base image or filesystem used to build the container (e.g., Docker, Library, or a local image).
`Bind path / Mount`	A directory on the host that is mapped (mounted) into the container at runtime.
`Overlay`	A writable filesystem layer that can be added to a read-only SIF image, allowing changes without modifying the original image.
`%post`	Definition file section where software installation commands are run at build time.
`%environment`	Definition file section that sets environment variables at runtime.
`%runscript`	Definition file section that defines the default command when the container is run.
`fakeroot`	A feature that allows an unprivileged user to operate with the appearance of root privileges inside the container, subject to host configuration and limitations.

2. Loading Apptainer on the RRC

Apptainer is available as an environment module on the RRC. Always load the module before using any Apptainer commands.

```
# Load Apptainer
module load apptainer

# Verify the version
apptainer --version

# List all available Apptainer versions
module avail apptainer

# Switch to a specific version if needed
module load apptainer/1.3.0

# Unload when finished
module unload apptainer
```

TIP: Add "module load apptainer" near the top of your SLURM job scripts to ensure Apptainer is available on the compute node.

3. Pulling and Using Existing Images

The fastest way to get started is to pull a pre-built image from a public registry such as Docker Hub or the NVIDIA GPU Cloud (NGC). Apptainer automatically converts Docker images to the SIF format.

3.1 Pulling from Docker Hub

```
# Pull a PyTorch image from Docker Hub (converts to .sif automatically)
apptainer pull docker://pytorch/pytorch:2.2.0-cuda11.8-cudnn8-runtime

# Pull an Ubuntu image
apptainer pull docker://ubuntu:22.04

# Pull and save with a custom name
apptainer pull --name my_pytorch.sif docker://pytorch/pytorch:2.2.0-cuda11.8-cudnn8-
runtime
```

3.2 Pulling from NVIDIA NGC

NVIDIA NGC provides GPU-optimized containers for deep learning and HPC workloads.

```
# Pull TensorFlow from NVIDIA NGC
apptainer pull docker://nvcr.io/nvidia/tensorflow:24.01-tf2-py3

# Pull PyTorch from NVIDIA NGC
apptainer pull docker://nvcr.io/nvidia/pytorch:24.01-py3
```

TIP: NVIDIA NGC images are heavily optimized for GPU workloads and are generally preferable to generic Docker Hub images for deep learning on the RRC.

3.3 Where to Store SIF Files

SIF files can be several gigabytes in size. Store them in shared storage, not your home directory.

Storage Location	Path	Recommended For
Primary project storage	`/account001/your_username/`	Long-term SIF storage, shared team images
Large data storage	`/data001/your_username/`	SIF files used in data-intensive jobs
Compute storage	`/compute002/your_username/`	SIF files used in active compute jobs
Home directory	`/home/your_username/`	NOT recommended — limited space

WARNING: Do not store SIF files in your home directory (/home/). Home directories have limited storage capacity and are not intended for large files.

4. Apptainer Definition Files

A definition file (also called a "def file") is a plain-text recipe that describes exactly how to build a container image. Using definition files is the recommended approach for reproducible, documented, and shareable research environments.

4.1 Definition File Structure

A definition file is divided into a header and a series of sections. The header defines the base image; the sections define what happens at build time and runtime.

Section	When It Runs	Purpose
`Header (Bootstrap / From)`	Build time — first	Defines the base image/source.
`%post`	Build time	Install software, run commands, set up the environment.
`%environment`	Runtime	Defines environment variables available inside the container.
`%runscript`	Runtime	Defines the default command executed by "apptainer run".
`%labels`	Build time	Add metadata such as author, version, description.
`%files`	Build time	Copy files/directories from the host into the container.
`%test`	Build time	Run commands to verify the container build.
`%help`	Runtime	Provides help text displayed by "apptainer run-help".

4.2 Minimal Definition File Example

The simplest possible definition file — an Ubuntu base image with Python 3 installed:

```
# minimal_python.def
Bootstrap: docker
From: ubuntu:22.04

%post
    apt-get update && apt-get install -y \
        python3 python3-pip && \
    apt-get clean

%environment
    export PATH=/usr/bin:$PATH

%runscript
    exec python3 "$@"

%labels
```

```
    Author your_username@fda.hhs.gov
    Version 1.0

%help
    A minimal Python 3 container.
    Usage: apptainer run minimal_python.sif script.py
```

4.3 Scientific Python Environment Example

A more complete example for data science and machine learning workloads:

```
# science_env.def

Bootstrap: docker
From: python:3.10-slim

%post
    python -m pip install --no-cache-dir \
        numpy pandas scipy scikit-learn \
        matplotlib seaborn jupyter \
        torch torchvision

%environment
    export OMP_NUM_THREADS=1

%runscript
    exec python "$@"

%labels
    Author your_username@fda.hhs.gov
    Version 1.0
    Description Scientific Python environment with PyTorch

%help
    Scientific Python container with NumPy, Pandas, Scikit-learn, and PyTorch.

    Usage:
        apptainer run science_env.sif my_script.py
```

4.4 GPU-Enabled Definition File Example

Building from an NVIDIA CUDA base image for GPU-accelerated workloads:

```
# gpu_pytorch.def

Bootstrap: docker
From: nvcr.io/nvidia/pytorch:24.01-py3

%post
    pip install --no-cache-dir \
        pandas scikit-learn matplotlib seaborn

%environment
    export PYTHONNOUSERSITE=1
    export OMP_NUM_THREADS=1

%runscript
    exec python "$@"

%labels
    Author your_username@fda.hhs.gov
    Version 1.0
```

```
    Description GPU-enabled PyTorch container based on NVIDIA NGC

%help
    GPU-enabled PyTorch container based on NVIDIA NGC.

    When running on a GPU node, use Apptainer's --nv option to make the host NVIDIA GPU
    and driver available to the container..

    Usage:
       apptainer run --nv gpu_pytorch.sif train.py
```

4.5 Copying Files Into the Container

Use the %files section to copy scripts or config files directly into the image at build time:

```
# copy_files.def

Bootstrap: docker
From: python:3.10-slim

%files
    /account001/your_username/scripts/setup.py  /opt/setup.py
    /account001/your_username/config/params.yaml /opt/params.yaml

%post
    python -m pip install --no-cache-dir \
    numpy pandas

    python /opt/setup.py

%runscript
    exec python "$@"
```

NOTE: Files copied via %files become part of the image at build time. For frequently changing data, use bind paths (Section 7) instead of %files.

5. Building Containers

Once you have a definition file, you can build it into a SIF image using `apptainer build`. On the RRC, users should use the `--fakeroot` flag when building containers. This allows users to perform many operations that normally require root privileges without actually having root access.

5.1 Basic Build Command

```
# Syntax
apptainer build --fakeroot <output_image.sif> <definition_file.def>

# Example: build the minimal Python image
apptainer build --fakeroot minimal_python.sif minimal_python.def

# Example: build the GPU PyTorch image
apptainer build --fakeroot gpu_pytorch.sif gpu_pytorch.def
```

IMPORTANT: Always use --fakeroot when building containers on the RRC. Building without it requires root access, which is not available to regular users on shared HPC systems.

5.2 Building on a Compute Node (Recommended)

Container builds can be resource-intensive. Build containers on a compute node through SLURM rather than on the login node.

Option A — Interactive build session

```
# Request an interactive CPU session
srun --pty --nodes=1 --ntasks=1 --cpus-per-task=8 --mem=32G \

    --time=02:00:00 bash -l

# Once on the compute node, load Apptainer
module load apptainer

# Set the temporary build directory
mkdir -p /tempfs001/$USER/apptainer

export APPTAINER_TMPDIR=/tempfs001/users/$USER/apptainer

# Build the container
apptainer build --fakeroot /account001/your_username/images/my_image.sif my_image.def
```

Option B — Batch build job (recommended for large images)

```
#!/bin/bash

#SBATCH --job-name=apptainer_build
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=02:00:00
#SBATCH --output=build_%j.out
#SBATCH --error=build_%j.err
#SBATCH --mail-user=your_email@fda.hhs.gov
#SBATCH --mail-type=END,FAIL

# Load Apptainer
module load apptainer

# Set the temporary build directory

Apptainer mkdir -p /tempfs001/users/$USER/apptainer

export APPTAINER_TMPDIR=/tempfs001/users/$USER/apptainer

# Build the container
apptainer build --fakeroot \
    /account001/your_username/images/my_image.sif \
    /account001/your_username/defs/my_image.def

echo "Build complete: $(date)"
```

TIP: Save your .def files in version control or a dedicated directory under /account001/your_username/defs/ so you can reproduce or update images later.

IMPORTANT: Apptainer uses temporary storage during the build process. On the RRC, use /tempfs001/users/$USER/apptainer for temporary build files. Store the final .sif image should be stored in the user's designated image directory under /account001

5.3 Build from a Container Registry (Without a Definition File)

You can also build a SIF directly from a container registry without creating a definition file:

```
# Build directly from Docker Hub
apptainer build –fakeroot \
  /account001/$USER/images/my_image.sif \
  docker://pytorch/pytorch:2.2.0-cuda11.8-cudnn8-runtime



# Build directly from NVIDIA NGC
apptainer build –fakeroot \
 /account001/$USER/images/tf_container.sif \
 docker://nvcr.io/nvidia/tensorflow:24.01-tf2-py3
```

5.4 Verifying the Build

After building the container, verify that the SIF image was created correctly and inspect its configuration.

```
# Check that the SIF image exists and view its size
ls -lh /account001/$USER/images/my_image.sif

# Inspect image metadata and labels
apptainer inspect /account001/$USER/images/my_image.sif

# View the runscript
apptainer inspect --runscript /account001/$USER/images/my_image.sif

# Display the help text
Apptainer run-help /account001/$USER/images/my_image.sif

# Run the %test section (if defined)
apptainer test /account001/$USER/images/my_image.sif

# Run the container to verify that it starts correctly
 apptainer run /account001/$USER/images/my_image.sif
```

6. Running Containers

Apptainer provides three main commands for running containers, each suited to different use cases.

6.1 apptainer exec — Run a Specific Command

Use apptainer exec to run a specific command inside the container. This is the most commonly used command in SLURM job scripts because it allows you to explicitly specify the application to run.

```
# Run a Python script inside the container
apptainer exec my_image.sif python3 my_script.py

# Run with GPU access
apptainer exec --nv gpu_pytorch.sif python train_model.py

# Run a shell command
apptainer exec my_image.sif bash -c "echo Hello from inside the container"

# Set up am environment variable inside the container
apptainer exec --env MY_VAR=hello my_image.sif python script.py
```

6.2 apptainer run — Execute the Runscript

Use `apptainer run` to execute the %runscript defined in the container image. This is useful when you want containers to behave like self-contained executable.

```
# Run the container runscript
apptainer run my_image.sif

# Pass arguments to the runscript
apptainer run my_image.sif my_script.py --epochs 50

# Run with GPU access
apptainer run --nv gpu_pytorch.sif train.py --batch-size 32
```

6.3 apptainer shell — Interactive Shell Inside the Container

Use `apptainer shell` to open an interactive bash session inside the container. This is useful for testing, debugging, and exploring the container before submitting batch jobs.

```
# Open an interactive shell
apptainer shell my_image.sif

# Shell with GPU access
apptainer shell --nv gpu_pytorch.sif

# Open a shell with a temporary writable filesystem
apptainer shell --writable-tmpfs my_image.sif
```

TIP: Use apptainer shell for interactive exploration and testing. Once your workflow is confirmed, switch to apptainer exec in a SLURM batch script for production runs.

6.4 Command Comparison

Command	What It Does	Best Used For
`apptainer exec <image> <cmd>`	Runs a specific command inside the container	SLURM job scripts, reproducible runs
`apptainer run <image>`	Runs the %runscript defined in the image	Self-contained container executables
`apptainer shell <image>`	Opens an interactive shell inside the container	Testing, debugging, exploration

7. Bind Paths and Storage Mounts

Apptainer containers provide an isolated environment, but selected host directories can be made available inside the container through bind paths (mounts). Bind paths allow applications running inside the container to access cluster storage at runtime.

Without access to the appropriate storage paths, a container may not be able to read input data or write output.

7.1 Default Bind Paths on the RRC

Apptainer provides an isolated environment, but selected host file systems can be made automatically available inside containers through **system-level bind paths** configured by the HPC administrators.

On the RRC, the following storage locations are configured as **default bind paths** and are automatically available inside Apptainer containers without requiring users to specify the --bind option.

Host Path	Available Inside Container	Purpose / Notes
/account001/	/account001/	Primary project storage and persistent user data
/data001/	/data001/	Shared storage for data-intensive workloads

For example, users can access their storage directly from an Apptainer container:

Access a project directory under /account001

```
apptainer exec my_image.sif \
    ls -la /account001/<your_project_directory>
```

If the project directory matches the user's username, $USER can be used:

```
apptainer exec my_image.sif \
    ls -la /account001/$USER
```

Access a personal directory under /data001:

```
apptainer exec my_image.sif \
    ls -la /data001/users/$USER
```

No --bind option is required because these paths are configured as system-level default bind paths.

NOTE: Other shared file systems, such as /compute001, /compute002, /ngs006, /ngs007, and /ngs008, are not configured as default bind paths. Users who need access to these file systems from inside a container must explicitly bind them using --bind. See **Section 7.2**.

7.2 Specifying Additional Bind Paths with --bind

Use the --bind flag to mount additional directories or to map a host path to a different path inside the container.

```
# Syntax
apptainer exec --bind <host_path>:<container_path> my_image.sif command

# Example: Bind /compute002
apptainer exec --bind /compute002/users/$USER:/data \ my_image.sif python3 script.py

#Inside the container, the host directory is accessible as: /data
```

7.4 Verifying Bind Paths Inside the Container

If you use --bind, you can verify that the directory is available inside the container:

```
apptainer exec --bind /compute002/users/$USER:/data \
    my_image.sif ls -la /data
```

8. Interactive Testing on a Compute Node

Before submitting a batch job, it is strongly recommended to test your container interactively on a compute node. This lets you catch errors early, verify storage access, and confirm GPU access before running a longer batch job.

8.1 Request an Interactive CPU Session

```
# Request an interactive CPU session
srun --pty \
    --nodes=1 \
    --cpus-per-task=4 \
    --mem=16G \
    --time=01:00:00 \
    bash -l

# Once on the compute node, load Apptainer
module load apptainer

# Open a shell inside your container
apptainer shell /account001/your_username/images/my_image.sif

# Test your application inside the container
python3 my_script.py
```

8.2 Request an Interactive GPU Session

```
# Request an interactive GPU session
srun --pty \
    --partition=<partition_name> \
    --gres=gpu:1 \
    --nodes=1 \
    --cpus-per-task=4 \
    --mem=32G \
    --time=01:00:00 \
    bash -l

# Load Apptainer and verify GPU is visible
module load apptainer
nvidia-smi

# Open a GPU-enabled shell inside your container
apptainer shell --nv /account001/your_username/images/gpu_pytorch.sif

# Inside the container, verify GPU access
python3 -c "import torch; print(torch.cuda.is_available())"
```

TIP: Always verify GPU access inside the container with a quick Python check before submitting a long training job. Use the --nv option when running a container that requires NVIDIA GPU access.

9. Running Containers in SLURM Batch Jobs

Once your container is tested interactively, submit it as a SLURM batch job using sbatch. All job scripts follow the same pattern: load the Apptainer module, then call apptainer exec or apptainer run with the appropriate flags.

9.1 CPU Batch Job

```
#!/bin/bash

#SBATCH --job-name=apptainer_cpu
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=04:00:00
#SBATCH --output=apptainer_cpu_%j.out
#SBATCH --error=apptainer_cpu_%j.err
#SBATCH --mail-user=your_email@fda.hhs.gov
#SBATCH --mail-type=END,FAIL

module load apptainer

IMAGE=/account001/your_username/images/science_env.sif
SCRIPT=/account001/your_username/scripts/analysis.py

echo "Job started on $(hostname) at $(date)"

apptainer exec $IMAGE python3 $SCRIPT \
    --input /data001/your_username/data
    --output /data001/your_username/results


echo "Job finished at $(date)"
```

9.2 GPU Batch Job

```
#!/bin/bash

#SBATCH --job-name=apptainer_gpu
#SBATCH --partition=<partition_name>
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=32G
#SBATCH --gres=gpu:1
#SBATCH --time=08:00:00
#SBATCH --output=apptainer_gpu_%j.out
#SBATCH --error=apptainer_gpu_%j.err
#SBATCH --mail-user=your_email@fda.hhs.gov
#SBATCH --mail-type=END,FAIL

module load apptainer

# Verify GPU allocation
nvidia-smi

IMAGE=/account001/your_username/images/gpu_pytorch.sif
SCRIPT=/account001/your_username/scripts/train_model.py

echo "Job started on $(hostname) at $(date)"

# --nv makes the NVIDIA GPU and required drivers available inside the container

apptainer exec --nv $IMAGE python3 $SCRIPT \
    --data /data001/users/$USER/dataset \
    --output /data001/users/$USER/model_output \
    --epochs 100 \
    --batch-size 32
```

```
echo "Job finished at $(date)"
```

10. Quick Reference

Task	Command
Load Apptainer	`module load apptainer`
Pull from Docker Hub	`apptainer pull docker://image:tag`
Pull from NVIDIA NGC	`apptainer pull docker://nvcr.io/nvidia/image:tag`
Build from definition file	`apptainer build --fakeroot image.sif image.def`
Run a command in container (CPU)	`apptainer exec image.sif python3 script.py`
Run a command in container (GPU)	`apptainer exec --nv image.sif python3 script.py`
Run the container runscript	`apptainer run image.sif`
Open interactive shell	`apptainer shell image.sif`
Open interactive shell (GPU)	`apptainer shell --nv image.sif`
Mount a directory	`apptainer exec --bind /host/path:/container/path image.sif cmd`
Set bind paths via env variable	`export APPTAINER_BINDPATH="/path1,/path2"`
Mount an additional directory	`apptainer exec --bind /host/path:/container/path image.sif command`
Inspect image metadata	`apptainer inspect image.sif`
View runscript	`apptainer inspect --runscript image.sif`
View help	`apptainer run-help image.sif`
Verify GPU access	`apptainer exec --nv image.sif nvidia-smi`

11. Additional Resources

For advanced features, configuration options, troubleshooting, and complete command documentation, please refer to the official Apptainer User Guide.

https://apptainer.org/docs/user/main/

12. Contacting Support

Channel	Details
Email	NCTRHPCSupport@fda.hhs.gov
Ticketing	Submit an ERIC ticket to ensure proper tracking and prioritization of support requests.